

3/2/25
Monday

Module-3 Trees & Graphs

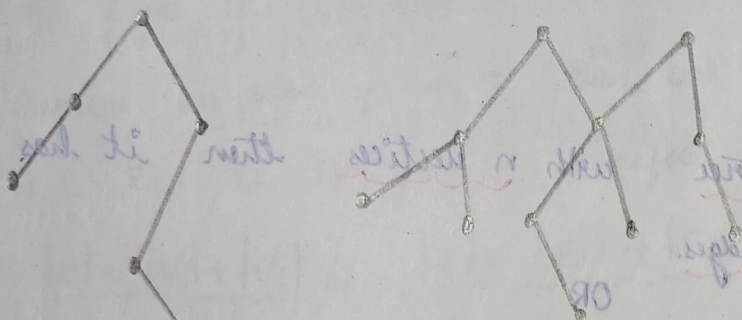
Algorithm

Trees:

Let $G = (V, E)$ be a loop free, undirected or directed graph then G is called a tree if it is connected & contains no cycles.

Trees = a connected, loop free, acyclic graph.

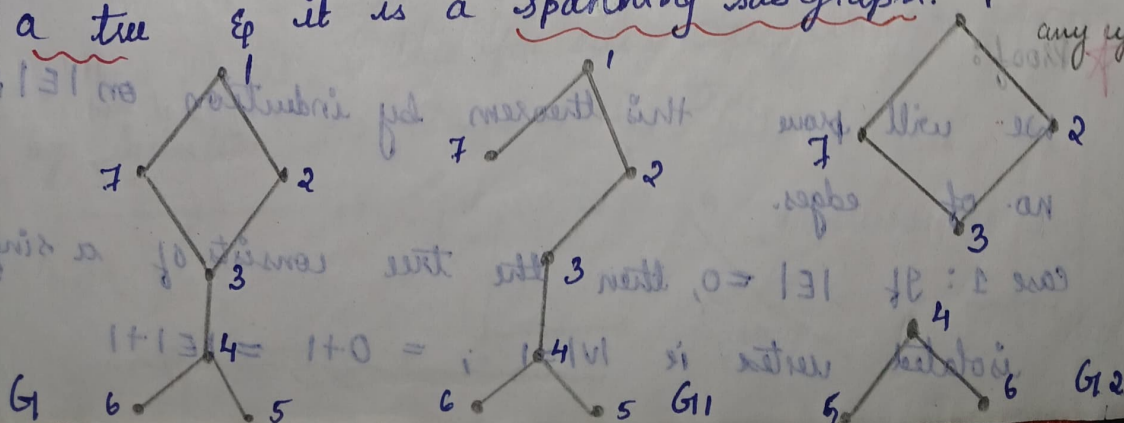
Eg:



Spanning tree: it includes all the vertices of G but with minimum no. of edges needed to keep the graph connected.

Let $G = (V, E)$ be an undirected graph, then a spanning tree is a subgraph of G , which itself forms a tree & it is a spanning subgraph without any cycles.

Eg:



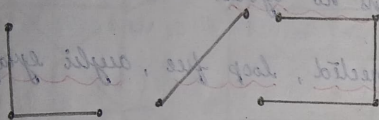
Here G_1 - is a spanning tree, G_2 - not a spanning tree since it is not connected & it contains cycle.

Forest:

A forest is an undirected graph, all of whose connected components are trees. In other words, a graph consisting of a disjoint union of trees.

Forest is an undirected, acyclic graph.

Eg:



Forest with 3 trees

4/2/25 Tuesday

Theorem:

→ If T is a tree with n vertices then it has precisely $n-1$ edges. OR

→ If $T = (V, E)$ be a tree then cardinality of V ,

$$|V| = |E| + 1.$$

★

Note:

$|V| = |E| + 1$ } cardinality of V , is the no. of vertices.

★

Proof:

We will prove this theorem by induction on $|E|$ ie no. of edges.

case 1: If $|E| = 0$, then the tree consists of a single isolated vertex ie $|V| = 1 = 0 + 1 = |E| + 1$

Case 2: when $|E| = 1$, then there are 2 vertices, since there is no loop, in T ie $|V| = 2 = 1 + 1 = |E| + 1$

Case 3: when $|E| = 2$, then there are 3 vertices, since there are no loops and parallel edges therefore, $|V| = 3 = 2 + 1 = |E| + 1$.

∴ the result is true for $|E| = 0, 1, 2$

Now assume that the theorem is true for every tree that contains at most k -edges, where $k \geq 0$.

Now consider a tree $T [V, E]$ that contains $k+1$ edges ie $|E| = k+1$.

Remove an edge from T then we get 2 subtrees $T_1 [V_1, E_1]$ & $T_2 [V_2, E_2]$ therefore cardinality of V ,

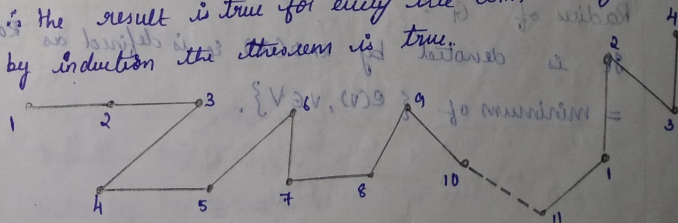
$$|V| = |V_1| + |V_2| \text{ \& } |E| = |E_1| + |E_2| + 1$$

Where $|E_1| \leq k$ & $|E_2| \leq k$. since T_1 & T_2 contains at most k edges then $|V_1| = |E_1| + 1$ & $|V_2| = |E_2| + 1$.

$$\text{Therefore } |V| = |V_1| + |V_2| = (|E_1| + 1) + (|E_2| + 1) = (|E_1| + |E_2| + 1) + 1$$

$$|V| = |E| + 1$$

∴ The result is true for every tree with $k+1$ edges ie by induction the theorem is true.



Distance, Eccentricity, Radius, Diameter & center of the graph.

→ Distance: min no. of edges you must cross to go from one vertex to another.
For any two vertices u & v connected by a path in $G(V, E)$. Then, we define the distance b/w u & v to be the length of the shortest uv path (ie the no. of edges in the shortest path) b/w them. And it is denoted by $d(u, v)$.

Note: $d(u, v) = \infty$.

→ $d(u, v) = \infty$, then there is no path connecting u & v .

→ In a tree since there is exactly 1 path b/w two vertices, the determination of distance is much easier.

Eccentricity of v :

→ Suppose V is the vertex set of G & v belongs to V , then eccentricity of v is denoted by $e(v)$ & is given by $e(v) = \text{Maximum of } \{d(u, v), u \in V\}$

↓
shortest path distance b/w v & v

Radius of G :

It is denoted by $Rad(G)$, & is defined as $Rad(G) = \text{minimum of } \{e(v), v \in V\}$.

minimum eccentricity of all vertices

Diameter of G :

It is denoted by $dia(G)$ & is defined by $dia(G) = \text{Maximum of } \{e(v), v \in V\}$

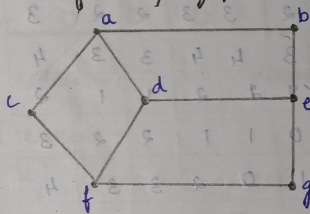
Center of G : vertex of graph G with min. eccentricity.

A vertex v is said to be a center of G if $e(v) = \text{radius of } G, rad(G)$

$$C(G) = \{v \in V : e(v) = \text{radius}(G)\}$$

5/2/25
Wednesday

Find the eccentricity, radius, diameter & center of the following graph.



Ans:	a	b	c	d	e	f	g
a	0	1	1	1	2	2	3
b	1	0	2	2	1	3	2
c	1	2	0	2	3	1	2
d	1	2	2	0	1	1	2
e	2	1	3	1	0	2	1
f	2	3	1	1	2	0	1
g	3	2	2	2	1	1	0

Eccentricity

$$e(a) = 3$$

$$e(b) = 3$$

$$e(c) = 3$$

$$e(d) = 2$$

$$e(e) = 3$$

$$e(f) = 3$$

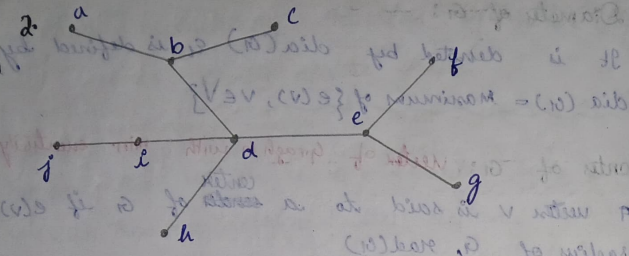
$$e(g) = 3$$

$$\text{radius} = \min_{\max} \{e(u, v)\} = 2$$

$$\text{Diameter} = \max \{e(u, v)\} = 3$$

$$\text{center of } G = \{d, e\}$$

11/2/25
Tuesday

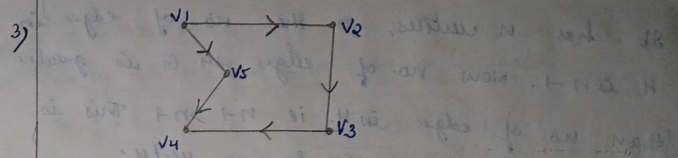


Ans:

	a	b	c	d	e	f	g	h	i	j
a	0	1	2	2	3	4	4	3	3	4
b	1	0	1	1	2	3	3	2	2	3
c	2	1	0	2	3	4	4	3	3	4
d	2	1	2	0	1	2	2	1	1	2
e	3	2	3	1	0	1	1	2	2	3
f	4	3	4	2	1	0	2	3	3	4
g	4	3	4	2	1	2	0	3	3	4
h	3	2	3	1	2	3	3	0	2	3
i	3	2	3	1	2	3	3	2	0	1
j	4	3	4	2	3	4	4	3	1	0

Eccentricity:
 $e(a) = 4$ $e(b) = 3$ $e(c) = 4$ $e(d) = 2$
 $e(e) = 3$ $e(f) = 4$ $e(g) = 4$ $e(h) = 3$
 $e(i) = 3$ $e(j) = 4$

Radius : min of eccentricity = 2 center = d
Diameter : Max of eccentricity = 4



Ans:

	v1	v2	v3	v4	v5
v1	0	1	2	2	1
v2	0	0	1	2	0
v3	0	0	0	1	0
v4	0	0	0	0	0
v5	0	0	0	1	0

radius = 0 diameter = 2 center = v5

Theorem:

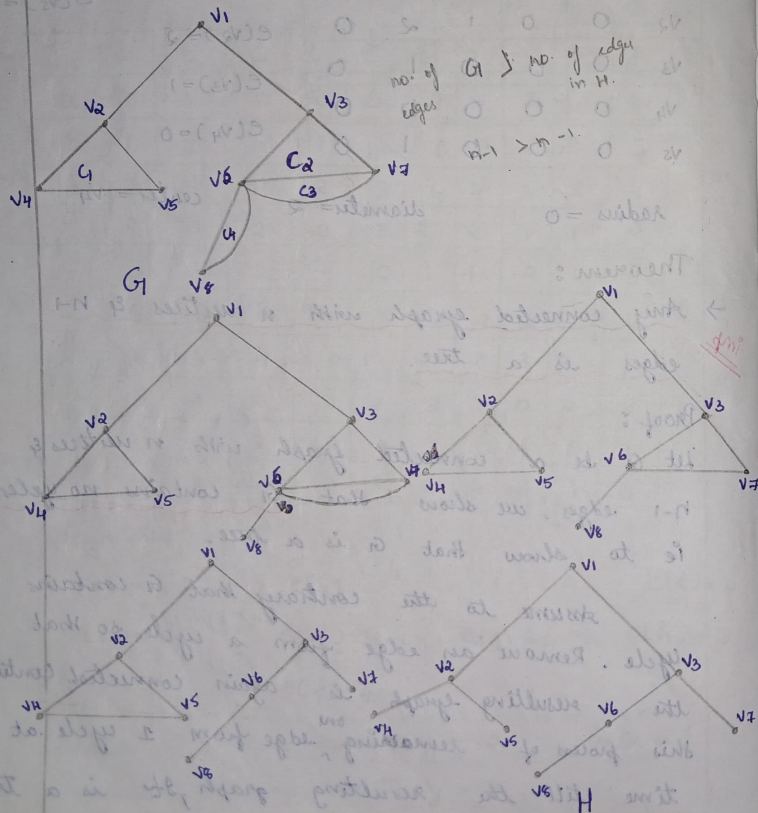
→ Any connected graph with n vertices & n-1 edges is a tree.

Proof:

Let G be a connected graph with n vertices & n-1 edges, we show that G contains no cycle.
 i.e. to show that G is a tree.

Assume to the contrary that G contains cycle. Remove an edge from a cycle so that the resulting graph is again connected. Continue this process of removing ^{one} edge from 1 cycle at a time till the resulting graph, H , is a tree.

It has n vertices, so the no. of edges in H is $n-1$. Now no. of edges in G is greater than no. of edges in H , i.e. $n-1 > n-1$. This is not possible & hence G has no cycles.
 $\therefore G$ is a tree.



isolates
Wednesday

Theorem:

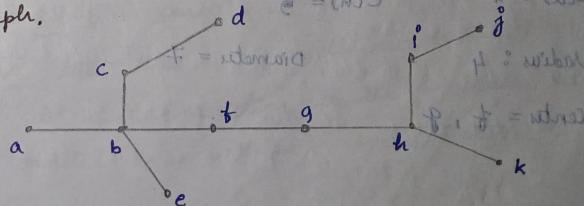
Every tree has either one or 2 centers.

Proof:

The maximum distance, $\max d(v, v_i)$ from a given vertex v to any other vertex occurs only when v_i is a pendant vertex. With this observation, let T be a tree having more than 2 vertices. Tree T has 2 or more pendant vertices.

Deleting all the pendant vertices from T , there is a resulting graph T_1 is again a tree. The removal of all pendant vertices from T uniformly reduces the eccentricities of the remaining vertices (T_1) by 1. Therefore the centers of T are also the centers of T_1 . From T_1 , we remove all pendant vertices & get another tree T_2 . Continuing this process, we either get a vertex which is a center of T or an edge where end vertices are the two centers of T .

1. Find eccentricity, radius, center & diameter of the graph.



Ans:

	a	b	c	d	e	f	g	h	i	j	k
a	0	1	2	3	2	2	3	4	5	6	5
b	1	0	1	2	1	1	2	3	4	5	4
c	2	1	0	1	2	2	3	4	5	6	5
d	3	2	1	0	3	4	5	6	6	7	6
e	2	1	2	3	0	2	3	4	5	6	5
f	2	1	2	4	2	0	1	2	3	4	3
g	3	2	3	4	3	1	0	1	2	3	2
h	4	3	4	5	4	2	1	0	1	2	1
i	5	4	5	6	5	3	2	1	0	1	2
j	6	5	6	7	6	4	3	2	1	0	3
k	5	4	5	6	5	3	2	1	2	3	0

eccentricity:

$$e(a) = 5$$

$$e(b) = 5$$

$$e(c) = 6$$

$$e(d) = 7$$

$$\text{radius} = 4$$

$$\text{centr} = f, g$$

$$e(e) = 6$$

$$e(f) = 4$$

$$e(g) = 4$$

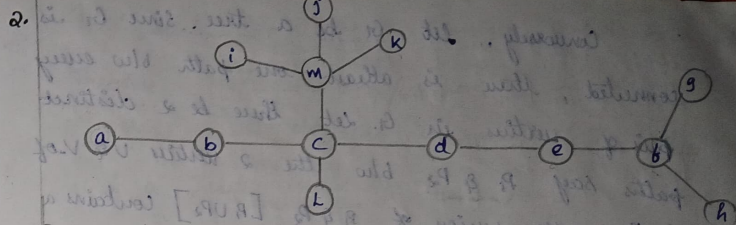
$$e(h) = 5$$

$$\text{Diameter} = 7$$

$$e(i) = 6$$

$$e(j) = 7$$

$$e(k) = 6$$



Ans: eccentricity:

$$e(a) = 5$$

$$e(b) = 5$$

$$e(c) = 6$$

$$\text{radius} = 3$$

$$e(d) = 7$$

$$e(f) = 4$$

$$e(g) = 4$$

$$\text{diameter} = 6$$

$$e(h) = 5$$

$$e(i) = 6$$

$$e(j) = 7$$

$$\text{centr} = d$$

$$e(e) = 6$$

$$e(k) = 6$$

$$e(l) = 5$$

$$e(m) = 5$$

11/2/25
Friday

Theorem:

A graph is a tree if & only if there is exactly one path between its every pair of its vertices.

Proof:

Let G be a graph & let there be exactly one path b/w every pair of vertices in G . So G is connected. If G contains a cycle, say b/w the vertices U & V then there are 2 distinct paths b/w U & V , which is a contradiction to the hypothesis. Hence G is connected & is without cycles, therefore G is a tree.

Conversely, let G be a tree. Since G is connected, there is at least one path b/w every pair of vertices in G . Let there be 2 distinct paths say P_1 & P_2 b/w the 2 vertices U & V of G . Then, the union of P_1 & P_2 [$P_1 \cup P_2$] contains a cycle which contradicts the fact that G is a tree. Hence there is exactly one path b/w every pair of vertices of a tree.

Theorem:

A vertex v in a tree is a cut vertex of tree, if & only if $d(v) \geq 2$.

Proof:

Let v be a cut vertex of a tree T . Since no pendant vertex of a tree can be its cut vertex, clearly we have $d(v) \geq 2$.

Conversely, let v be a vertex of a tree T , such that $d(v) \geq 2$. Then v is called a internal vertex of T . Since $d(v) \geq 2$, so there are at least two neighbours for v in T . Let U & W be two neighbours of v . Then $U-v-W$ is a $U-W$ path in T . We know that we have

the path ' $U-v-W$ ' is a unique ' $U-W$ ' path in T . Therefore $T-v$ [removing v from T] is disconnected & U & W are in different components of T . Therefore v is a cut vertex of T .

18/12/2021
Tuesday

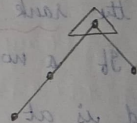
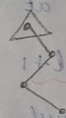
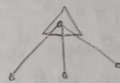
Rooted tree: one node is designated as the root.

A tree in which one vertex is distinguished from all other vertices is called a rooted tree.

In a diagram of a rooted tree, the root is generally marked distinctly. We will show the root enclosed

in a small triangle.

Eg: rooted tree with 4 vertices.



Generally, term tree means without any roots.

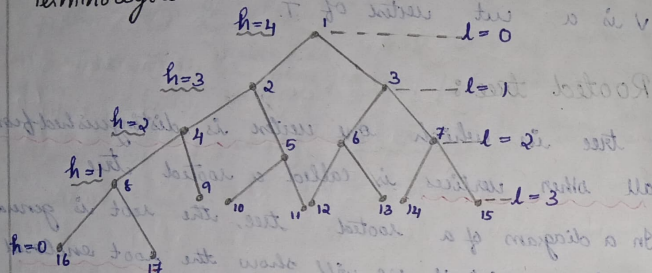
However, for emphasis they are sometimes called free trees [or non-rooted trees] to differentiate them from the rooted kind.

Binary tree:

It is a special class of rooted tree. Binary tree can be defined as finite set of nodes such that

every vertex is of degree 1 or 3, except one vertex [the vertex of degree 2].

Terminologies:



level:

level is the rank in hierarchy. The root node is at level 0. If a node is at level l then:

→ Its child is at level $l+1$

→ And its parent is at level $l-1$.

Height of a node: Height of tree with n nodes is $\frac{n-1}{2}$.

→ It is the no. of edges on the longest path from the node to the leaf.

→ A leaf node will have height zero.

Height of a tree:

→ It would be the height of its root node + 1.

Degree of a node:

It is the no. of children it has.

Parent node:

Immediate predecessor of a node.

child node:

All immediate successor of a node.

Root node:

Node does not having any parent.

leaf node:

Node does not having any child.

Siblings:

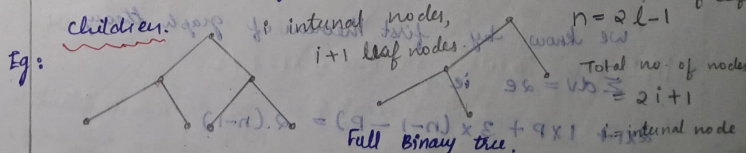
children of same parent.

Internal vertex:

A non-leaf vertex in a tree is called internal vertex.

Full Binary tree:

A binary tree is a full binary tree if it contains maximum possible no. of nodes at all levels. OR we can say each node has either 0 or 2 children.





not a binary tree.

Q) The no. of vertices in a binary tree is always odd.

Ans: Let T be a binary tree with n vertices, then T has only one vertex of degree 2 & remaining $n-1$ vertices are of degree 3 or 1.

We know that no. of odd vertices in a graph should be in even no. Therefore $n-1$ is even.

Thus n is odd, i.e. the no. of vertices in a binary tree is always odd.

19/2/25
Wednesday

Q) Let T be a binary tree with n vertices show that the no. of pendant vertices in T is $n+1$.

Ans: Let T be a binary tree with n vertices, then T has only one vertex of degree 2, & the remaining $n-1$ vertices are of degree 1 or 3.

Let P be the no. of pendant vertices in T then the no. of vertices of degree 3 is $n-1-P$.

We know by first theorem of graph theory

$$\sum d_v = 2e \quad \text{ie}$$

$$2x + 1xP + 3 \times (n-1-P) = 2(n-1) \quad (\text{since } T \text{ is a tree with } n \text{ vertices, } n-1 \text{ edges})$$

$$2 + P + 3n - 3 - 3P = 2n - 2$$

$$2 + 2 - 2P + 3n - 2n - 3 = 0$$

$$1 - 2P + n = 0$$

$$\text{or } n+1 = 2P$$

$$\frac{n+1}{2} = P$$

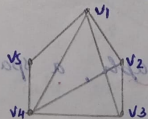
Q) Every connected graph has a spanning tree.

Ans: Let G be a connected graph of n vertices. Pick an arbitrary edge of G & name it as e_1 . If e_1 belongs to a cycle of G , then delete it from G , [else leave it unchanged & pick another one]. Let $H = G - e_1$. Now choose an edge e_2 of H . If e_2 belongs to a cycle of H then remove e_2 from H .

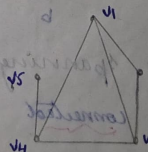
Proceed the step until all cycles in G are removed successively. Since G is a finite graph, the procedure terminates after a no. of times. At this stage we get

a subgraph $T(G)$, none of whose edges belongs to a cycle. Therefore T is a connected, acyclic subgraph of G on n vertices. And hence is a spanning tree of G .

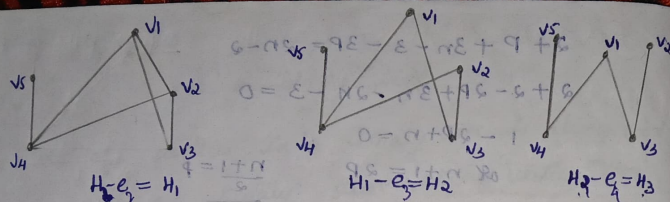
Hence the proof.



G_1



$G_1 - e_1 = H$



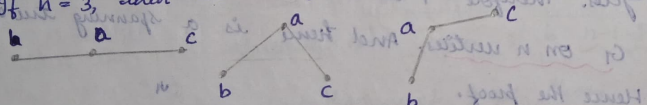
Counting trees:

Arthur Cayley, introduced no. of trees that one can construct with 'n' distinct vertices. A graph in which each vertex is assigned a unique name or label [i.e. no two vertices have the same label] is called labelled trees.

Cayley's theorem:

The no. of labelled trees with n vertices ($n \geq 2$) is n^{n-2} . If $n=4$, then the no. of labelled trees are $4^{4-2} = 4^2 = 16$.

If $n=3$, then the no. of labelled trees are $3^{3-2} = 3$.



Minimum spanning tree [MST]:

G is a connected undirected graph, a spanning

tree of G with the smallest total weight is called a minimum spanning tree [MST]. STW connect all vertices with least possible total weight. For constructing MST we use the following edge weight.

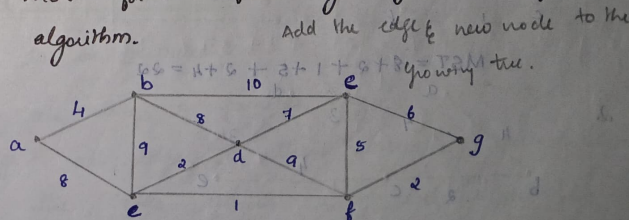
Algorithms:

- PRIM'S Algorithm
- KRUSKAL'S Algorithm
- DIJKSTRA'S Algorithm
- FLOYD - WARSHALL Algorithm.

1. PRIM'S ALGORITHM:

Steps to be followed:

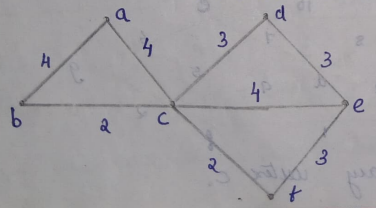
1. Choose a vertex at random.
 2. Select unvisited vertex to the visited vertex (vertex chosen at random) choose a smallest weighted edge from the visited vertex that connect to another node that is not visited.
 3. Repeat step-2 until all vertices are visited.
- Find MST for the following graphs using Prim's algorithm.



Ans: Take the arbitrary vertex c .

Corresponding graph	visited vertex (uv)	unvisited vertex (v/w)
	{c}	{a, b, d, e, f, g}
	{c, d}	{a, b, e, f, g}
	{c, d, f}	{a, b, e, g}
	{c, d, f, g}	{a, b, e}
	{c, d, f, g, e}	{a, b}
	{c, d, f, g, e, b}	{a}
	{a, b, c, d, e, g, f}	{}

$MST = 8 + 2 + 1 + 5 + 2 + 4 = 22$



Corresponding graph	visited vertex	unvisited vertex
	{c}	{a, b, d, e, f}
	{c, b}	{a, d, e, f}
	{b, c, f, e}	{a, d}
	{a, b, c, f, e}	{d}
	{a, b, c, d, e, f}	{}

$MST = 4 + 2 + 2 + 3 + 3 = 14$

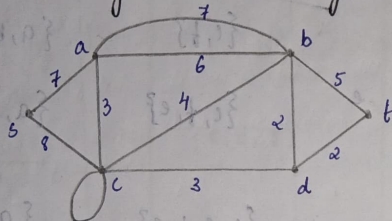
2. KRUSHKAL'S ALGORITHM:

Steps to be followed: select the smallest weight edge that does not form a cycle.

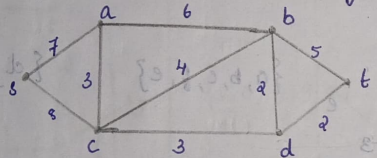
1. Remove all loops & parallel edges.
2. Arrange all edges in their increasing order of weight.
3. Add the edge which has least weight by checking there is no cycle. If cycle formed, discard it.

4. Continue step 3 until it is connected one. Or

Q. Find mst using kruskal's algorithm: $V=1$ edges.

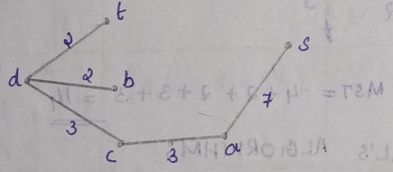


Ans: Removing loops & parallel edges.

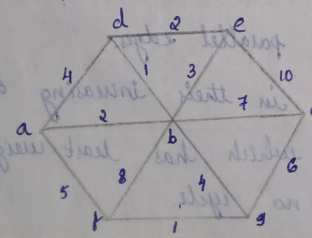


Arranging edges in its increasing weight.

db	dt	cd	ca	cb	bt	ab	as	sc
2	3	3	3	4	5	6	7	8

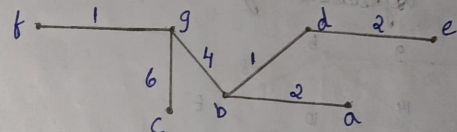


$$MST = 2 + 2 + 3 + 3 + 7 = 17$$



Ans: Arranging all edges in its increasing weight

fg	db	ab	de	be	ad	bg	af	cg
1	1	2	2	3	4	4	5	6
bc	bf	ec						
7	8	10						



$$MST = 1 + 1 + 2 + 3 + 4 + 2 = 16$$

3. DIJKSTRA'S ALGORITHM:

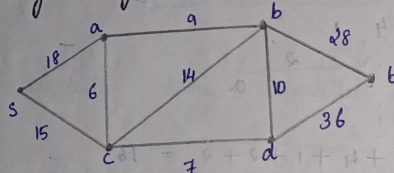
Dijkstra's algorithm is a graph algorithm that finds shortest path between a starting node & all other node. The steps are:

1. Initialize distances: start from source node.
2. Set distances of starting node to zero and all other nodes to infinity.
2. Explore neighbours: Find all nodes that are adjacent to current node and calculate their distances to the end.
3. Visit the shortest node: choose the unvisited node with the smallest distance as the current node.

4. Repeat :

Repeat steps 2 & 3 until the shortest distance to the end node is found.

Q. Find the shortest distance from vertex 's' to vertex 't' by using dijkstra's algorithm for the following graph.



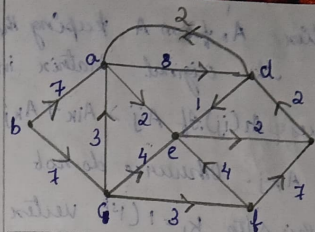
Ans:

[illegible]

$s - a - b - t$

$$m_{st} = 55$$

2.



$b^{(0)}$	$a^{(0)}$	$c^{(0)}$	$d^{(0)}$	$e^{(0)}$	$f^{(0)}$	$g^{(0)}$
$b^{(1)}$	$a^{(1)}$	$c^{(1)}$	$d^{(1)}$	$e^{(1)}$	$f^{(1)}$	$g^{(1)}$
$b^{(2)}$	$a^{(2)}$	$c^{(2)}$	$d^{(2)}$	$e^{(2)}$	$f^{(2)}$	$g^{(2)}$
$b^{(3)}$	$a^{(3)}$	$c^{(3)}$	$d^{(3)}$	$e^{(3)}$	$f^{(3)}$	$g^{(3)}$
$b^{(4)}$	$a^{(4)}$	$c^{(4)}$	$d^{(4)}$	$e^{(4)}$	$f^{(4)}$	$g^{(4)}$
$b^{(5)}$	$a^{(5)}$	$c^{(5)}$	$d^{(5)}$	$e^{(5)}$	$f^{(5)}$	$g^{(5)}$
$b^{(6)}$	$a^{(6)}$	$c^{(6)}$	$d^{(6)}$	$e^{(6)}$	$f^{(6)}$	$g^{(6)}$
$b^{(7)}$	$a^{(7)}$	$c^{(7)}$	$d^{(7)}$	$e^{(7)}$	$f^{(7)}$	$g^{(7)}$
$b^{(8)}$	$a^{(8)}$	$c^{(8)}$	$d^{(8)}$	$e^{(8)}$	$f^{(8)}$	$g^{(8)}$
$b^{(9)}$	$a^{(9)}$	$c^{(9)}$	$d^{(9)}$	$e^{(9)}$	$f^{(9)}$	$g^{(9)}$
$b^{(10)}$	$a^{(10)}$	$c^{(10)}$	$d^{(10)}$	$e^{(10)}$	$f^{(10)}$	$g^{(10)}$
$b^{(11)}$	$a^{(11)}$	$c^{(11)}$	$d^{(11)}$	$e^{(11)}$	$f^{(11)}$	$g^{(11)}$
$b^{(12)}$	$a^{(12)}$	$c^{(12)}$	$d^{(12)}$	$e^{(12)}$	$f^{(12)}$	$g^{(12)}$
$b^{(13)}$	$a^{(13)}$	$c^{(13)}$	$d^{(13)}$	$e^{(13)}$	$f^{(13)}$	$g^{(13)}$
$b^{(14)}$	$a^{(14)}$	$c^{(14)}$	$d^{(14)}$	$e^{(14)}$	$f^{(14)}$	$g^{(14)}$
$b^{(15)}$	$a^{(15)}$	$c^{(15)}$	$d^{(15)}$	$e^{(15)}$	$f^{(15)}$	$g^{(15)}$
$b^{(16)}$	$a^{(16)}$	$c^{(16)}$	$d^{(16)}$	$e^{(16)}$	$f^{(16)}$	$g^{(16)}$
$b^{(17)}$	$a^{(17)}$	$c^{(17)}$	$d^{(17)}$	$e^{(17)}$	$f^{(17)}$	$g^{(17)}$
$b^{(18)}$	$a^{(18)}$	$c^{(18)}$	$d^{(18)}$	$e^{(18)}$	$f^{(18)}$	$g^{(18)}$
$b^{(19)}$	$a^{(19)}$	$c^{(19)}$	$d^{(19)}$	$e^{(19)}$	$f^{(19)}$	$g^{(19)}$
$b^{(20)}$	$a^{(20)}$	$c^{(20)}$	$d^{(20)}$	$e^{(20)}$	$f^{(20)}$	$g^{(20)}$
$b^{(21)}$	$a^{(21)}$	$c^{(21)}$	$d^{(21)}$	$e^{(21)}$	$f^{(21)}$	$g^{(21)}$
$b^{(22)}$	$a^{(22)}$	$c^{(22)}$	$d^{(22)}$	$e^{(22)}$	$f^{(22)}$	$g^{(22)}$
$b^{(23)}$	$a^{(23)}$	$c^{(23)}$	$d^{(23)}$	$e^{(23)}$	$f^{(23)}$	$g^{(23)}$
$b^{(24)}$	$a^{(24)}$	$c^{(24)}$	$d^{(24)}$	$e^{(24)}$	$f^{(24)}$	$g^{(24)}$
$b^{(25)}$	$a^{(25)}$	$c^{(25)}$	$d^{(25)}$	$e^{(25)}$	$f^{(25)}$	$g^{(25)}$
$b^{(26)}$	$a^{(26)}$	$c^{(26)}$	$d^{(26)}$	$e^{(26)}$	$f^{(26)}$	$g^{(26)}$
$b^{(27)}$	$a^{(27)}$	$c^{(27)}$	$d^{(27)}$	$e^{(27)}$	$f^{(27)}$	$g^{(27)}$
$b^{(28)}$	$a^{(28)}$	$c^{(28)}$	$d^{(28)}$	$e^{(28)}$	$f^{(28)}$	$g^{(28)}$
$b^{(29)}$	$a^{(29)}$	$c^{(29)}$	$d^{(29)}$	$e^{(29)}$	$f^{(29)}$	$g^{(29)}$
$b^{(30)}$	$a^{(30)}$	$c^{(30)}$	$d^{(30)}$	$e^{(30)}$	$f^{(30)}$	$g^{(30)}$
$b^{(31)}$	$a^{(31)}$	$c^{(31)}$	$d^{(31)}$	$e^{(31)}$	$f^{(31)}$	$g^{(31)}$
$b^{(32)}$	$a^{(32)}$	$c^{(32)}$	$d^{(32)}$	$e^{(32)}$	$f^{(32)}$	$g^{(32)}$
$b^{(33)}$	$a^{(33)}$	$c^{(33)}$	$d^{(33)}$	$e^{(33)}$	$f^{(33)}$	$g^{(33)}$
$b^{(34)}$	$a^{(34)}$	$c^{(34)}$	$d^{(34)}$	$e^{(34)}$	$f^{(34)}$	$g^{(34)}$
$b^{(35)}$	$a^{(35)}$	$c^{(35)}$	$d^{(35)}$	$e^{(35)}$	$f^{(35)}$	$g^{(35)}$
$b^{(36)}$	$a^{(36)}$	$c^{(36)}$	$d^{(36)}$	$e^{(36)}$	$f^{(36)}$	$g^{(36)}$
$b^{(37)}$	$a^{(37)}$	$c^{(37)}$	$d^{(37)}$	$e^{(37)}$	$f^{(37)}$	$g^{(37)}$
$b^{(38)}$	$a^{(38)}$	$c^{(38)}$	$d^{(38)}$	$e^{(38)}$	$f^{(38)}$	$g^{(38)}$
$b^{(39)}$	$a^{(39)}$	$c^{(39)}$	$d^{(39)}$	$e^{(39)}$	$f^{(39)}$	$g^{(39)}$
$b^{(40)}$	$a^{(40)}$	$c^{(40)}$	$d^{(40)}$	$e^{(40)}$	$f^{(40)}$	$g^{(40)}$
$b^{(41)}$	$a^{(41)}$	$c^{(41)}$	$d^{(41)}$	$e^{(41)}$	$f^{(41)}$	$g^{(41)}$
$b^{(42)}$	$a^{(42)}$	$c^{(42)}$	$d^{(42)}$	$e^{(42)}$	$f^{(42)}$	$g^{(42)}$
$b^{(43)}$	$a^{(43)}$	$c^{(43)}$	$d^{(43)}$	$e^{(43)}$	$f^{(43)}$	$g^{(43)}$
$b^{(44)}$	$a^{(44)}$	$c^{(44)}$	$d^{(44)}$	$e^{(44)}$	$f^{(44)}$	$g^{(44)}$
$b^{(45)}$	$a^{(45)}$	$c^{(45)}$	$d^{(45)}$	$e^{(45)}$	$f^{(45)}$	$g^{(45)}$
$b^{(46)}$	$a^{(46)}$	$c^{(46)}$	$d^{(46)}$	$e^{(46)}$	$f^{(46)}$	$g^{(46)}$
$b^{(47)}$	$a^{(47)}$	$c^{(47)}$	$d^{(47)}$	$e^{(47)}$	$f^{(47)}$	$g^{(47)}$
$b^{(48)}$	$a^{(48)}$	$c^{(48)}$	$d^{(48)}$	$e^{(48)}$	$f^{(48)}$	$g^{(48)}$
$b^{(49)}$	$a^{(49)}$	$c^{(49)}$	$d^{(49)}$	$e^{(49)}$	$f^{(49)}$	$g^{(49)}$
$b^{(50)}$	$a^{(50)}$	$c^{(50)}$	$d^{(50)}$	$e^{(50)}$	$f^{(50)}$	$g^{(50)}$
$b^{(51)}$	$a^{(51)}$	$c^{(51)}$	$d^{(51)}$	$e^{(51)}$	$f^{(51)}$	$g^{(51)}$
$b^{(52)}$	$a^{(52)}$	$c^{(52)}$	$d^{(52)}$	$e^{(52)}$	$f^{(52)}$	$g^{(52)}$
$b^{(53)}$	$a^{(53)}$	$c^{(53)}$	$d^{(53)}$	$e^{(53)}$	$f^{(53)}$	$g^{(53)}$
$b^{(54)}$	$a^{(54)}$	$c^{(54)}$	$d^{(54)}$	$e^{(54)}$	$f^{(54)}$	$g^{(54)}$
$b^{(55)}$	$a^{(55)}$	$c^{(55)}$	$d^{(55)}$	$e^{(55)}$	$f^{(55)}$	$g^{(55)}$
$b^{(56)}$	$a^{(56)}$	$c^{(56)}$	$d^{(56)}$	$e^{(56)}$	$f^{(56)}$	$g^{(56)}$
$b^{(57)}$	$a^{(57)}$	$c^{(57)}$	$d^{(57)}$	$e^{(57)}$	$f^{(57)}$	$g^{(57)}$
$b^{(58)}$	$a^{(58)}$	$c^{(58)}$	$d^{(58)}$	$e^{(58)}$	$f^{(58)}$	$g^{(58)}$
$b^{(59)}$	$a^{(59)}$	$c^{(59)}$	$d^{(59)}$	$e^{(59)}$	$f^{(59)}$	$g^{(59)}$
$b^{(60)}$	$a^{(60)}$	$c^{(60)}$	$d^{(60)}$	$e^{(60)}$	$f^{(60)}$	$g^{(60)}$
$b^{(61)}$	$a^{(61)}$	$c^{(61)}$	$d^{(61)}$	$e^{(61)}$	$f^{(61)}$	$g^{(61)}$
$b^{(62)}$	$a^{(62)}$	$c^{(62)}$	$d^{(62)}$	$e^{(62)}$	$f^{(62)}$	$g^{(62)}$
$b^{(63)}$	$a^{(63)}$	$c^{(63)}$	$d^{(63)}$	$e^{(63)}$	$f^{(63)}$	$g^{(63)}$
$b^{(64)}$	$a^{(64)}$	$c^{(64)}$	$d^{(64)}$	$e^{(64)}$	$f^{(64)}$	$g^{(64)}$
$b^{(65)}$	$a^{(65)}$	$c^{(65)}$	$d^{(65)}$	$e^{(65)}$	$f^{(65)}$	$g^{(65)}$
$b^{(66)}$	$a^{(66)}$	$c^{(66)}$	$d^{(66)}$	$e^{(66)}$	$f^{(66)}$	$g^{(66)}$
$b^{(67)}$	$a^{(67)}$	$c^{(67)}$	$d^{(67)}$	$e^{(67)}$	$f^{(67)}$	$g^{(67)}$
$b^{(68)}$	$a^{(68)}$	$c^{(68)}$	$d^{(68)}$	$e^{(68)}$	$f^{(68)}$	$g^{(68)}$
$b^{(69)}$	$a^{(69)}$	$c^{(69)}$	$d^{(69)}$	$e^{(69)}$	$f^{(69)}$	$g^{(69)}$
$b^{(70)}$	$a^{(70)}$	$c^{(70)}$	$d^{(70)}$	$e^{(70)}$	$f^{(70)}$	$g^{(70)}$
$b^{(71)}$	$a^{(71)}$	$c^{(71)}$	$d^{(71)}$	$e^{(71)}$	$f^{(71)}$	$g^{(71)}$
$b^{(72)}$	$a^{(72)}$	$c^{(72)}$	$d^{(72)}$	$e^{(72)}$	$f^{(72)}$	$g^{(72)}$
$b^{(73)}$	$a^{(73)}$	$c^{(73)}$	$d^{(73)}$	$e^{(73)}$	$f^{(73)}$	$g^{(73)}$
$b^{(74)}$	$a^{(74)}$	$c^{(74)}$	$d^{(74)}$	$e^{(74)}$	$f^{(74)}$	$g^{(74)}$
$b^{(75)}$	$a^{(75)}$	$c^{(75)}$	$d^{(75)}$	$e^{(75)}$	$f^{(75)}$	$g^{(75)}$
$b^{(76)}$	$a^{(76)}$	$c^{(76)}$	$d^{(76)}$	$e^{(76)}$	$f^{(76)}$	$g^{(76)}$
$b^{(77)}$	$a^{(77)}$	$c^{(77)}$	$d^{(77)}$	$e^{(77)}$	$f^{(77)}$	$g^{(77)}$
$b^{(78)}$	$a^{(78)}$	$c^{(78)}$	$d^{(78)}$	$e^{(78)}$	$f^{(78)}$	$g^{(78)}$
$b^{(79)}$	$a^{(79)}$	$c^{(79)}$	$d^{(79)}$	$e^{(79)}$	$f^{(79)}$	$g^{(79)}$
$b^{(80)}$	$a^{(80)}$	$c^{(80)}$	$d^{(80)}$	$e^{(80)}$	$f^{(80)}$	$g^{(80)}$
$b^{(81)}$	$a^{(81)}$	$c^{(81)}$	$d^{(81)}$	$e^{(81)}$	$f^{(81)}$	$g^{(81)}$
$b^{(82)}$	$a^{(82)}$	$c^{(82)}$	$d^{(82)}$	$e^{(82)}$	$f^{(82)}$	$g^{(82)}$
$b^{(83)}$	$a^{(83)}$	$c^{(83)}$	$d^{(83)}$	$e^{(83)}$	$f^{(83)}$	$g^{(83)}$
$b^{(84)}$	$a^{(84)}$	$c^{(84)}$	$d^{(84)}$	$e^{(84)}$	$f^{(84)}$	$g^{(84)}$
$b^{(85)}$	$a^{(85)}$	$c^{(85)}$	$d^{(85)}$	$e^{(85)}$	$f^{(85)}$	$g^{(85)}$
$b^{(86)}$	$a^{(86)}$	$c^{(86)}$	$d^{(86)}$	$e^{(86)}$	$f^{(86)}$	$g^{(86)}$
$b^{(87)}$	$a^{(87)}$	$c^{(87)}$	$d^{(87)}$	$e^{(87)}$	$f^{(87)}$	$g^{(87)}$
$b^{(88)}$	$a^{(88)}$	$c^{(88)}$	$d^{(88)}$	$e^{(88)}$	$f^{(88)}$	$g^{(88)}$
$b^{(89)}$	$a^{(89)}$	$c^{(89)}$	$d^{(89)}$	$e^{(89)}$	$f^{(89)}$	$g^{(89)}$
$b^{(90)}$	$a^{(90)}$	$c^{(90)}$	$d^{(90)}$	$e^{(90)}$	$f^{(90)}$	$g^{(90)}$
$b^{(91)}$	$a^{(91)}$	$c^{(91)}$	$d^{(91)}$	$e^{(91)}$	$f^{(91)}$	$g^{(91)}$
$b^{(92)}$	$a^{(92)}$	$c^{(92)}$	$d^{(92)}$	$e^{(92)}$	$f^{(92)}$	$g^{(92)}$
$b^{(93)}$	$a^{(93)}$	$c^{(93)}$	$d^{(93)}$	$e^{(93)}$	$f^{(93)}$	$g^{(93)}$
$b^{(94)}$	$a^{(94)}$	$c^{(94)}$	$d^{(94)}$	$e^{(94)}$	$f^{(94)}$	$g^{(94)}$
$b^{(95)}$	$a^{(95)}$	$c^{(95)}$	$d^{(95)}$	$e^{(95)}$	$f^{(95)}$	$g^{(95)}$
$b^{(96)}$	$a^{(96)}$	$c^{(96)}$	$d^{(96)}$	$e^{(96)}$	$f^{(96)}$	$g^{(96)}$
$b^{(97)}$	$a^{(97)}$	$c^{(97)}$	$d^{(97)}$	$e^{(97)}$	$f^{(97)}$	$g^{(97)}$
$b^{(98)}$	$a^{(98)}$	$c^{(98)}$	$d^{(98)}$	$e^{(98)}$	$f^{(98)}$	$g^{(98)}$
$b^{(99)}$	$a^{(99)}$	$c^{(99)}$	$d^{(99)}$	$e^{(99)}$	$f^{(99)}$	$g^{(99)}$
$b^{(100)}$	$a^{(100)}$	$c^{(100)}$	$d^{(100)}$	$e^{(100)}$	$f^{(100)}$	$g^{(100)}$

b-a-e-g-d

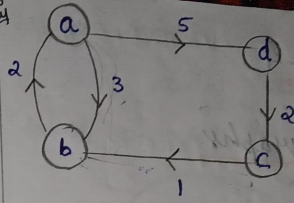
$$b \rightarrow a =$$
$$b \rightarrow c = 7$$
$$b \rightarrow c + f = 7 + 3 = 10$$
$$b \rightarrow a \rightarrow e \rightarrow g \rightarrow d = 13$$
$$b \rightarrow a \rightarrow e = a$$
$$b \rightarrow a \rightarrow e \rightarrow g = 11$$

4. FLOYD-WARSHALL ALGORITHM:

steps to be followed :

1. Construct an adjacency matrix A with all the costs of edges present in the graph. If there is no path b/w 2 vertices mark the value as ∞ .

- | | | | |
|---|---|---|---|
| 4 | 2 | 3 | 4 |
| 0 | 3 | 5 | 6 |
| 5 | 0 | 2 | 3 |
| 3 | 6 | 0 | 1 |
| 4 | 2 | 5 | 7 |



$A(d) =$

	a	b	c	d
a	0	3	7	5
b	2	0	9	7
c	3	4	0	8
d	5	3	8	0